Atelier Git & GitHub — Mise a niveau avec le projet AfriTalent

---

Par Robert DIASSÉ

Niveau : L1 Web

Duree estimee : 2h30

Objectifs

A la fin de cet atelier, vous serez capable de :

- Installer et configurer Git sur votre machine
- Creer un depot GitHub et le lier a votre projet local
- Utiliser les commandes fondamentales de Git
- Realiser les deux premiers commits reglementaires du projet AfriTalent

---

Prerequis

- Connexion Internet disponible
- Un compte GitHub cree sur https://github.com (gratuit)
- Visual Studio Code installe

Partie 1 — Comprendre Git et GitHub

Qu'est-ce que Git ?

Git est un systeme de controle de version. Il garde une trace de toutes les modifications apportees a vos fichiers, comme un journal horodate et infalsifiable. Grace a Git, vous pouvez :

- Revenir a une version anterieure en cas d'erreur
- Voir qui a modifie quoi et quand

- Travailler a plusieurs sans ecraser le travail des autres

Git fonctionne entierement en local sur votre machine. Il n'a pas besoin d'Internet.

Qu'est-ce que GitHub ?

GitHub est une plateforme en ligne qui heberge des depots Git. Il vous permet de :

- Sauvegarder votre code sur un serveur distant
- Partager et presenter votre travail (portfolio)
- Publier un site web gratuitement via GitHub Pages

Resume : Git est l'outil de versionnage, GitHub est le serveur de stockage en ligne.

Partie 2 — Installation et configuration de Git

Etape 1 — Installer Git

Rendez-vous sur https://git-scm.com/downloads et telechargez la version correspondant a votre systeme.

Sous Windows : lancez l'installateur et conservez tous les parametres par defaut. Cochez "Git Bash Here" si l'option est proposee.

Sous Linux (Ubuntu/Debian) :

```
sudo apt update
sudo apt install git
```

Point de controle : ouvrez un terminal (Git Bash sous Windows, Terminal sous Linux/Mac) et tapez :

```
git --version
```

Resultat attendu (le numero peut varier) :

```
git version 2.44.0
```

Si ce message s'affiche, Git est correctement installe. Sinon, relancez l'installation.

Etape 2 — Configurer votre identite

Git associe chaque modification a un auteur. Cette configuration se fait une seule fois sur votre machine.

```
git config --global user.name "Prenom Nom"
git config --global user.email "votre@email.com"
```

Utilisez votre vrai prenom, nom, et le meme email que votre compte GitHub.

Point de controle : verifiez la configuration :

```
git config --list
```

Votre nom et votre email doivent apparaitre dans la liste. Si c'est le cas, passez a la suite.

Partie 3 — Creer le depot GitHub du projet

Etape 3 — Creer le depot sur GitHub

1. Connectez-vous sur https://github.com
2. Cliquez sur le bouton "+" en haut a droite, puis "New repository"
3. Remplissez les champs comme suit :
   - Repository name : `NOM-Prenom-AfriTalent` (remplacez par votre nom et prenom reels)
   - Description : `Projet fil rouge — Plateforme AfriTalent`
   - Visibilite : cochez "Public"
   - Ne cochez PAS "Add a README file" (vous le ferez vous-meme)
4. Cliquez sur "Create repository"

GitHub affiche une page avec des instructions. Gardez cette page ouverte, vous en aurez besoin a l'etape 9.

Partie 4 — Creer l'arborescence du projet en local

Etape 4 — Creer le dossier du projet

Sur votre machine, creez un dossier en lui donnant exactement le meme nom que votre depot GitHub.
Exemple :

```
Diasse-Robert-AfriTalent/
```

Ouvrez ce dossier dans VS Code : menu Fichier > Ouvrir le dossier.

Etape 5 — Creer tous les fichiers et dossiers

Dans l'explorateur de VS Code, creez la structure suivante. Pour l'instant, tous les fichiers sont vides :

```
Diasse-Robert-AfriTalent/
├── index.html
├── freelances.html
├── tarifs.html
├── about.html
```

```
├── contact.html
├── css/
│   └── style.css
├── js/
│   └── main.js
├── README.md
└── .gitignore
```

Pour creer un dossier dans VS Code : clic droit dans l'explorateur > "New Folder". Nommez-le `css`, puis faites de meme pour `js`.

Remplissez le fichier `.gitignore` avec ce contenu (il indique a Git quels fichiers ignorer) :

```
.DS_Store
Thumbs.db
node_modules/
```

Etape 6 — Ajouter la structure HTML de base dans chaque page

Chaque fichier `.html` doit contenir la structure minimale suivante. Voici le modele pour `index.html` :

```html
<!DOCTYPE html>
<html lang="fr">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>AfriTalent — Accueil</title>

    <!-- Bootstrap 5 via CDN : feuille de style -->
    <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css"
rel="stylesheet">

    <!-- Votre feuille de style personnelle -->
    <link rel="stylesheet" href="css/style.css">
</head>
<body>

    <!-- Le contenu de la page sera ajoute ici -->

    <!-- Bootstrap 5 via CDN : script JavaScript -->
    <script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/js/bootstrap.bundle.min.js"
></script>

    <!-- Votre script JavaScript -->
    <script src="js/main.js"></script>
```

```
</body>
</html>
```

Reproduisez cette structure dans `freelances.html`, `tarifs.html`, `about.html` et `contact.html`. Modifiez uniquement la valeur de la balise `<title>` pour chaque page.

Ajoutez dans `README.md` :

```
# AfriTalent

Projet fil rouge — Plateforme de mise en relation entre freelances africains et clients.

Auteur : Prenom NOM
Promotion : L2 Web — ISI
```

Point de controle : ouvrez `index.html` dans votre navigateur. La page doit s'ouvrir sans erreur (elle est vide pour l'instant, c'est normal).

Partie 5 — Les commandes Git essentielles

Lisez ce tableau avant de continuer. Vous y reviendrez si vous avez un doute.

Commande	Role
`git init`	Initialise un depot Git dans le dossier courant
`git status`	Affiche l'etat des fichiers (modifies, suivis, non suivis)
`git add <fichier>`	Ajoute un fichier specifique a la zone de preparation
`git add .`	Ajoute TOUS les fichiers modifies a la zone de preparation
`git commit -m "message"`	Enregistre les modifications avec un message descriptif
`git remote add origin <url>`	Lie le depot local au depot GitHub
`git push -u origin main`	Envoie les commits vers GitHub (premiere fois)
`git push`	Envoie les commits suivants vers GitHub
`git log --oneline`	Affiche l'historique resume des commits

Le cycle de travail Git se resume ainsi :

```
Modifier les fichiers  -->  git add .  -->  git commit -m "..."  -->  git push
```

Partie 6 — Initialiser Git et realiser le premier commit

Etape 7 — Initialiser le depot local

Dans VS Code, ouvrez le terminal integre (menu Terminal > New Terminal). Verifiez que vous etes bien dans votre dossier de projet, puis tapez :

```
git init
```

Resultat attendu :

```
Initialized empty Git repository in /chemin/vers/Diasse-Robert-AfriTalent/.git/
```

Verifiez l'etat de votre depot :

```
git status
```

Git liste tous vos fichiers en rouge. Cela signifie qu'ils existent mais ne sont pas encore suivis par Git.

Etape 8 — Realiser le COMMIT 1

Ajoutez tous les fichiers a la zone de preparation :

```
git add .
```

Verifiez que tous les fichiers sont maintenant en vert :

```
git status
```

Enregistrez le commit avec le message exact impose par les consignes du projet :

```
git commit -m "Init : création du dépôt, arborescence des fichiers et structure
HTML de base"
```

Resultat attendu :

```
[main (root-commit) a1b2c3d] Init : création du dépôt, arborescence des fichiers
et structure HTML de base
 9 files changed, 45 insertions(+)
```

Le code `a1b2c3d` est l'identifiant unique de votre commit (le votre sera different).

Etape 9 — Lier le depot local a GitHub et pousser

Retournez sur la page GitHub de votre depot. Copiez l'URL HTTPS de votre depot. Elle ressemble a :

```
https://github.com/votre-username/Diasse-Robert-AfriTalent.git
```

Dans le terminal, tapez les trois commandes suivantes dans l'ordre :

```
# 1. On lie le depot local au depot GitHub
git remote add origin https://github.com/votre-username/Diasse-Robert-
AfriTalent.git

# 2. On s'assure que la branche principale s'appelle "main"
git branch -M main

# 3. On envoie le commit vers GitHub (-u memorise la destination pour les push
suivants)
git push -u origin main
```

Si GitHub vous demande vos identifiants, entrez votre nom d'utilisateur GitHub et votre mot de passe (ou token d'acces personnel si demande).

Point de controle : rafraichissez la page de votre depot sur GitHub. Vous devez voir :

- Tous vos fichiers listes dans le depot
- Le message du commit visible en haut de la liste
- Le fichier README.md affiche automatiquement sur la page d'accueil du depot

Si tout cela s'affiche, le COMMIT 1 est valide.

Partie 7 — Completer le HTML et realiser le deuxieme commit

Etape 10 — Ajouter la structure HTML complete dans chaque page

Vous allez maintenant remplir chaque page avec la structure semantique et la navbar Bootstrap. Voici le modele complet pour `index.html` :

```
<!DOCTYPE html>
<html lang="fr">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>AfriTalent — Accueil</title>
    <link
```

```
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css"
rel="stylesheet">
    <link rel="stylesheet" href="css/style.css">
</head>
<body>

    <!-- Navigation Bootstrap commune a toutes les pages -->
    <nav class="navbar navbar-expand-lg navbar-dark bg-dark">
        <div class="container">
            <a class="navbar-brand" href="index.html">AfriTalent</a>
            <button class="navbar-toggler" type="button"
                    data-bs-toggle="collapse"
                    data-bs-target="#navMenu"
                    aria-controls="navMenu"
                    aria-expanded="false"
                    aria-label="Toggle navigation">
                <span class="navbar-toggler-icon"></span>
            </button>
            <div class="collapse navbar-collapse" id="navMenu">
                <ul class="navbar-nav ms-auto">
                    <li class="nav-item"><a class="nav-link active"
href="index.html">Accueil</a></li>
                    <li class="nav-item"><a class="nav-link"
href="freelances.html">Freelances</a></li>
                    <li class="nav-item"><a class="nav-link"
href="tarifs.html">Tarifs</a></li>
                    <li class="nav-item"><a class="nav-link" href="about.html">A
propos</a></li>
                    <li class="nav-item"><a class="nav-link"
href="contact.html">Contact</a></li>
                </ul>
            </div>
        </div>
    </nav>

    <!-- Contenu principal -->
    <main>

        <!-- Section Hero : accroche principale -->
        <section id="hero">
            <div class="container py-5">
                <h1>Trouvez le talent africain qu'il vous faut</h1>
                <p>AfriTalent connecte les freelances africains aux clients du
monde entier.</p>
                <a href="freelances.html" class="btn btn-primary">Decouvrir les
talents</a>
            </div>
        </section>

        <!-- Section Services -->
        <section id="services">
            <div class="container py-5">
                <h2>Nos domaines d'expertise</h2>
                <div class="row">
```

```
                    <div class="col-md-4">
                        <h3>Developpement Web</h3>
                        <p>Sites vitrine, applications web, e-commerce.</p>
                    </div>
                    <div class="col-md-4">
                        <h3>Design Graphique</h3>
                        <p>Identite visuelle, UI/UX, illustrations.</p>
                    </div>
                    <div class="col-md-4">
                        <h3>Marketing Digital</h3>
                        <p>Reseaux sociaux, SEO, campagnes publicitaires.</p>
                    </div>
                </div>
            </div>
        </section>

        <!-- Section Statistiques -->
        <section id="statistiques">
            <div class="container py-5">
                <h2>AfriTalent en chiffres</h2>
                <div class="row text-center">
                    <div class="col-md-4"><p>1 200+ Freelances</p></div>
                    <div class="col-md-4"><p>45 Pays representes</p></div>
                    <div class="col-md-4"><p>3 800+ Projets realises</p></div>
                </div>
            </div>
        </section>

        <!-- Section Temoignages -->
        <section id="temoignages">
            <div class="container py-5">
                <h2>Ce que disent nos clients</h2>
                <blockquote class="blockquote">
                    <p>AfriTalent m'a permis de trouver un developpeur competent
en moins de 48h.</p>
                    <footer class="blockquote-footer">Aminata Diallo, Directrice
Marketing</footer>
                </blockquote>
            </div>
        </section>

    </main>

    <!-- Pied de page -->
    <footer>
        <div class="container py-4">
            <p>&copy; 2025 AfriTalent. Tous droits reserves.</p>
        </div>
    </footer>

    <script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/js/bootstrap.bundle.min.js"
></script>
    <script src="js/main.js"></script>
```

```
</body>
</html>
```

Reproduisez une structure equivalente (avec un contenu adapte) pour chaque page :

- `freelances.html` : section avec des cards de profils freelance (nom, competence, tarif)
- `tarifs.html` : section avec un tableau ou des cards presentant les forfaits
- `about.html` : section sur l'histoire d'AfriTalent, la mission, l'equipe
- `contact.html` : section avec un formulaire (champs Nom, Email, Message, bouton Envoyer)

La navbar doit etre identique dans toutes les pages. Seul l'attribut `active` change sur le lien de la page courante.

Point de controle : ouvrez chaque page dans le navigateur. La navbar Bootstrap doit s'afficher correctement et les liens de navigation doivent pointer vers les bonnes pages.

Etape 11 — Realiser le COMMIT 2

Verifiez l'etat de votre depot :

```
git status
```

Git doit lister vos 5 fichiers HTML comme modifies. Ajoutez toutes les modifications :

```
git add .
```

Enregistrez le commit avec le message exact impose :

```
git commit -m "HTML : structure complète des 5 pages avec sections et contenu
texte"
```

Envoyez le commit vers GitHub :

```
git push
```

Point de controle : sur GitHub, cliquez sur le lien "commits" en haut de la liste des fichiers. Vous devez voir exactement 2 commits dans l'historique :

```
HTML : structure complète des 5 pages avec sections et contenu texte
Init : création du dépôt, arborescence des fichiers et structure HTML de base
```

Pour verifier en local :

```
git log --oneline
```

Resultat attendu :

```
b4c5d6e HTML : structure complète des 5 pages avec sections et contenu texte
a1b2c3d Init : création du dépôt, arborescence des fichiers et structure HTML de
base
```

Recapitulatif des commandes utilisees

```
git init                              Initialise le depot local
git status                            Verifie l'etat des fichiers
git add .                             Prepare tous les fichiers
git commit -m "message"               Enregistre un commit
git remote add origin <url>           Lie le depot local a GitHub
git branch -M main                    Renomme la branche principale en "main"
git push -u origin main               Premier envoi vers GitHub
git push                              Envois suivants
git log --oneline                     Historique resume des commits
```

Points d'amelioration (a realiser seul apres l'atelier)

1. Activer GitHub Pages pour publier votre site en ligne : dans les parametres de votre depot sur GitHub > Pages > Source : branche "main". Votre site sera accessible a l'adresse `https://votre-username.github.io/NOM-Prenom-AfriTalent/`.

2. Creer une branche de developpement pour travailler sans toucher la branche principale : `git checkout -b dev`. Quand votre travail est pret, fusionnez via une Pull Request sur GitHub.

3. Completer le `README.md` avec une capture d'ecran du site, la liste des technologies utilisees et les instructions pour lancer le projet localement.

4. Explorer `git diff` pour voir les modifications en cours avant un commit : `git diff index.html`.